

Table 1

<code>.help</code>	Lists special commands
<code>.save <filename></code>	Saves the session inputs into a file, to be loaded later. Saving into file makes the commands you tried persistent. This helps review expressions outside of the Node prompt. Note that even the failed/erroneous expressions are saved into the file. You can remove the erroneous input using an editor like Vi or Notepad.
<code>.load <filename></code>	An earlier saved file can be executed at the current prompt.
<code>.break</code>	While trying JavaScript compound expressions or conditional statements, you might be lost in closing braces. This command helps you come out of the current statement and start fresh in a new line.

The command line parser is a REPL (Read Eval Print Loop). The few commands given in Table 1 help while trying Node REPL.

Node Package Manager

Node packages are modular libraries that help in specific functionalities. For example, file system management such as listing files in a directory and reading files is provided by the 'fs' package. This makes Node programming easy where you can pick and reuse the existing code snippet in your code, rather than coding from scratch. For example, to write a robust Web application that takes an input, processes it and gets back output to the browser, could take around 100 lines of code. But with existing packages, you could do it in a few lines.

Node Package Manager (npm) allows users to install, update and uninstall packages to the Node environment. The best part is that the installation and use of packages is very simple. If you can think of new functionality, it is very easy to create and add a new package. Currently, there are 40,000+ packages for Node in the repository!

To install a new package you can use:

```
$ npm install <package-name> //to install a package and
                             //dependent packages
$ npm ls                      //to list all the installed
                             //and dependent packages
```

Node Package Manager maintains the packages and dependencies learnt from a JSON file. When you install a package, the dependent packages are picked automatically for installation.

For a list of all packages that can be installed from the npm registry, visit: <https://npmjs.org>

Writing command line utilities using Node

Node can be used to write shell scripts using JavaScript. If you are a Web programmer with JavaScript experience, you

need not learn any other shell scripting. Given below is the procedure to make a JavaScript file execute like shell script from the command prompt.

The shell script below is written to watch for any changes in a file. Whenever the watched file content is changed or modified, the user gets a line printed with the size and time of modification.

```
1 #!/usr/bin/env node
2
3 var filename = "";
4 if (process.argv.length == 2)
5 {   6   console.log("Syntax: ./watch-file.js
    <filename>");
7   return 1;
8 }
9 else
10 {
11   filename = process.argv[2];
12   console.log("Filename being watched: " + filename);
13 }
14
15 var fs = require('fs');
16
17 fs.watchFile(filename, function (curr, prev) {
18   console.log(filename + ": (size:" + curr.size+ ") " +
    curr.mtime);
19 });
```

Here's an explanation of the script given above.

Line 1: Instruction to use the Node environment to execute the script.

Line 3 - 13: Accept the file to watch as command line argument. If the file name is not sent as a command line parameter, print the syntax. If the file name is entered, print <name of the file> is being watched for modifications.

Line 15: Use the Node file system library.

Line 17: `watchFile` is a file system method. The first